

LAPORAN PRAKTIKUM
PEMOGRAMAN ALGORITMA PEMOGRAMAN

PERULANGAN WHILE

disusun Oleh:

Khaula Lathifa Firdausyi

NIM 2511531007

Dosen Pengampu: DR. Wahyudi, S.T, M.T

Asisten Praktikum: Aufan Taufiqurrahman



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2025

DAFTAR ISI

DAFTAR ISI	i
KATA PENGANTAR.....	ii
BAB I	1
PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	2
BAB II.....	3
PEMBAHASAN	3
2.1 Dasar Teori	3
2.2 Praktikum “PerulanganWhile1”.....	4
2.3 Praktikum “Lempardadu”	5
2.4 Praktikum “GamePenjumlahan”	6
2.5 Praktikum “SentinelLoop”	7
BAB III	10
KESIMPULAN.....	10
3.1 Kesimpulan	10
DAFTAR PUSTAKA	11

KATA PENGANTAR

Laporan praktikum ini disusun sebagai bentuk pertanggungjawaban atas pelaksanaan kegiatan praktikum mata kuliah Algoritma Pemograman yang membahas tentang Perulangan While di Java. Melalui laporan ini penulis dapat lebih memahami materi praktikum dan dengan penulisan laporan ini dapat melatih ketelitian, keteraturan, serta kemampuan menulis sesuai kaidah akademik pada tingkat dasar. Dengan demikian, laporan praktikum yang dihasilkan dapat berfungsi sebagai media pembelajaran, dokumentasi kegiatan, sekaligus sarana untuk melatih keterampilan menulis ilmiah yang akan bermanfaat dalam pembelajaran selanjutnya.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat terbuka terhadap segala bentuk kritik dan saran yang bersifat membangun, demi perbaikan kualitas laporan maupun pemahaman penulis di laporan berikutnya.

Padang, 2025

Penulis

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan perangkat lunak, struktur kendali perulangan (looping) merupakan salah satu konsep dasar yang sangat penting. Perulangan memungkinkan eksekusi blok kode secara berulang selama kondisi tertentu terpenuhi, sehingga mengurangi pengulangan kode dan meningkatkan efisiensi program. Pemahaman mendalam tentang berbagai jenis perulangan—seperti `while`, `do-while`, serta penggunaan sentinel dan flag—menjadi fondasi penting dalam menyusun logika program yang dinamis dan responsif terhadap input pengguna.

Pada praktikum ini, akan mempelajari penerapan perulangan dalam berbagai skenario pemrograman. Beberapa contoh implementasi meliputi pembuatan game sederhana berbasis penjumlahan angka acak dengan sistem poin (`GamePenjumlahan`), simulasi lempar dadu yang berhenti saat jumlah tertentu tercapai (`Lempardadu`), validasi input seperti password menggunakan perulangan `do-while`, serta pemanfaatan sentinel loop untuk menjumlahkan deret angka hingga nilai penanda (misalnya 0) dimasukkan (`SentinelLoop`). Selain itu, penggunaan variabel boolean sebagai flag untuk mengontrol alur perulangan (`PerulanganWhile1`) memberikan gambaran nyata tentang bagaimana logika kompleks dapat disusun secara modular dan terkendali.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum ini antara lain sebagai berikut:

1. Memahami cara kerja perulangan `while` dan `do-while` dalam Java.
2. Menerapkan perulangan untuk menangani input pengguna, simulasi sederhana, dan pengolahan data berulang.
3. Menggunakan sentinel dan flag untuk mengatur kapan perulangan dimulai atau dihentikan.
4. Menggabungkan perulangan dengan `Scanner` dan `Random` agar program lebih interaktif.

5. Melatih logika berpikir melalui pemecahan masalah berbasis perulangan.

1.3 Manfaat Praktikum

Manfaat dari praktikum ini antara lain sebagai berikut:

1. Mengetahui kapan dan bagaimana perulangan bisa menyederhanakan kode.
2. Dapat membuat program yang responsif, seperti game kecil atau sistem validasi.
3. Lebih terbiasa mengendalikan alur program secara dinamis—sesuai kebutuhan, bukan hanya urutan kaku.
4. Lebih lancar memadukan struktur kendali dengan fitur Java dasar.
5. Membangun dasar logika yang berguna untuk pemrograman lanjutan.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Perulangan (looping) merupakan salah satu struktur kendali fundamental dalam pemrograman yang memungkinkan suatu bagian kode dieksekusi berulang kali selama kondisi tertentu masih terpenuhi. Dalam bahasa pemrograman Java, dua bentuk perulangan yang sering digunakan adalah while dan do-while. Perulangan while mengevaluasi kondisi di awal sebelum memutuskan apakah blok kode di dalamnya akan dijalankan, jika kondisi langsung bernilai false, maka blok tersebut tidak akan dieksekusi sama sekali. Sebaliknya, do-while menjamin bahwa blok kode dijalankan minimal satu kali karena pengecekan kondisi dilakukan setelah eksekusi blok. Kedua bentuk ini sangat berguna dalam berbagai skenario pemrograman, terutama ketika jumlah pengulangan tidak diketahui di awal dan bergantung pada input atau keadaan dinamis selama program berjalan. Salah satu pola umum dalam penerapan perulangan adalah sentinel loop, yaitu perulangan yang berhenti ketika pengguna memasukkan nilai khusus (disebut sentinel value), seperti angka 0 atau -1, yang tidak diproses sebagai data biasa melainkan sebagai sinyal akhir. Selain itu, penggunaan variabel flag biasanya berupa variabel boolean, memungkinkan pengendalian alur perulangan secara lebih fleksibel, misalnya, flag dapat diubah nilainya berdasarkan kondisi tertentu dalam program untuk menghentikan atau melanjutkan perulangan. Agar program menjadi interaktif, perulangan sering dikombinasikan dengan kelas Scanner untuk membaca input dari pengguna secara berulang, serta kelas Random untuk menghasilkan nilai acak dalam simulasi seperti lempar dadu atau kuis matematika.

2.2 Praktikum “PerulanganWhile1”

```
package pekan6_2511531007;

import java.util.Scanner;

public class PerulanganWhile1_2511531007 {

    public static void main(String[] args) {
        int counter = 0;
        String jawab;
        boolean running = true;
        //deklarasi scanner
        Scanner scan = new Scanner(System.in);
        while (running) {
            counter++;
            System.out.println("Jumlah = "+counter);
            System.out.print("Apakah lanjut (ya / tidak?)");
            jawab = scan.nextLine();
            //cek jawab = tidak, perulangan berhenti
            if (jawab.equalsIgnoreCase("tidak")) {
                running = false;
            }
        }
        System.out.println("Anda sudah melakukan perulangan sebanyak "+counter+" kali");
    }
}
```

Gambar 2.1 kode program praktikum “PerulanganWhile1”

Kode program di atas menggunakan menggunakan variabel boolean sebagai flag untuk mengontrol perulangan. Berikut ini adalah penjelasan mengenai kode program di atas:

1. Impor java.util.Scanner.
2. Buat kelas PerulanganWhile1 dan metode main.
3. Inisialisasi Scanner dan variabel:
 - boolean *running* = true;
 - String jawab;
 - Int *counter* = 0;
4. Gunakan perulangan while (*running*):
 - Tambah nilai counter dengan 1;
 - Tampilkan output, “Jumlah = “ + counter (nilai counter yang sudah ditambah 1 sebelumnya);
 - Tampilkan pesan, “Apakah Lanjut? (ya/tidak): ”
 - Baca input: jawab = scan.nextLine();
 - Jika input sama dengan "tidak", *running* = false;.
5. Setelah loop berhenti, tampilkan pesan “Anda sudah melakukan perulangan sebanyak ” + counter + “ kali “

Dari kode program diatas akan menghasilkan output:

```
Jumlah = 1
Apakah lanjut (ya / tidak?)ya
Jumlah = 2
Apakah lanjut (ya / tidak?)ya
Jumlah = 3
Apakah lanjut (ya / tidak?)ya
Jumlah = 4
Apakah lanjut (ya / tidak?)tidak
Anda sudah melakukan perulangan sebanyak 4 kali
```

Gambar 2.2 Output kode program “PerulanganWhile1”

2.3 Praktikum “Lempardadu”

```
package pekan6_2511531007;

import java.util.Random;

public class Lempardadu_2511531007 {

    public static void main(String[] args) {
        Random rand = new Random();
        int tries = 0;
        int sum = 0;
        while (sum != 7) {
            //roll the dice once
            int dadu1 = rand.nextInt(6) + 1;
            int dadu2 = rand.nextInt(6) + 1;
            sum = dadu1 + dadu2;
            System.out.println(dadu1 + " + " + dadu2 + " = " + sum);
            tries++;
        }
        System.out.println("You won after " + tries + " tries!");
    }
}
```

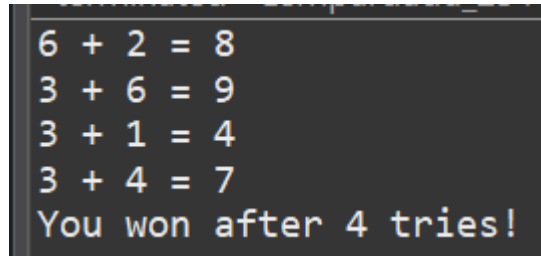
Gambar 2.3 kode program praktikum “Lempardadu”

Kode ini menggunakan perulangan while, karena program perlu terus melempar dua dadu selama jumlahnya belum sama dengan 7. Kondisi `sum != 7` diperiksa sebelum setiap iterasi. Jika kondisi bernilai true (jumlah belum 7), blok kode dijalankan; jika sudah false (jumlah = 7), perulangan berhenti. Berikut ini adalah penjelasan mengenai kode program di atas:

1. Impor `java.util.Random`
2. Buat kelas `Lempardadu`
3. Dalam `main`, buat objek `Random`
4. Inisialisasi `tries = 0` dan `sum = 0`
5. Gunakan `while (sum != 7)`, apabila nilai `sum=7` program akan berhenti
6. Di dalam loop:
 - Hasilkan dua angka acak 1–6 (`dadu1`, `dadu2`)
 - Hitung `sum = dadu1 + dadu2`
 - Tampilkan hasil lemparan
 - Tambah `tries` dengan 1

7. Setelah loop, tampilkan jumlah percobaan

Dari kode program diatas akan menghasilkan output:



```
6 + 2 = 8
3 + 6 = 9
3 + 1 = 4
3 + 4 = 7
You won after 4 tries!
```

Gambar 2.4 Output kode program “Lempardadu”

2.4 Praktikum “GamePenjumlahan”

```
package pekan6_2511531007;

import java.util.Random;
import java.util.Scanner;

public class GamePenjumlahan_2511531007 {

    public static void main(String[] args) {
        Scanner console = new Scanner(System.in);
        Random rand = new Random();
        //paly until user gets 3 wrong
        int points = 0;
        int wrong = 0;
        while (wrong < 3) {
            int result = play(console, rand); //play one game
            if (result > 0) {
                points++;
            } else {
                wrong++;
            }
        }
        System.out.println("You earned " + points + " total points.");
    }

    //membuat soal penjumlahan dan ditampilkan ke user
    public static int play(Scanner console, Random rand) {
        // print the operands being added, and sum them
        int operands = rand.nextInt(4) + 2;
        int sum = rand.nextInt(10) + 1;
        System.out.print(sum);
        for (int i = 2; i <= operands; i++) {
            int n = rand.nextInt(10) + 1;
            sum += n;
            System.out.print(" + " + n);
        }
        System.out.print(" = ");

        //read user's guess and report whether it was correct
        int guess = console.nextInt();
        if (guess == sum) {
            return 1;
        } else {
            System.out.println("Wrong! The answer was " + sum);
            return 0;
        }
    }
}
```

Gambar 2.5 kode program praktikum “GamePenjumlahan”

Kode program ini menggunakan kombinasi perulangan while dan for untuk membuat kuis matematika interaktif. Perulangan while di metode main mengontrol alur utama game, permainan terus berjalan selama pengguna belum salah sebanyak tiga kali. Setiap putaran, metode play() dipanggil untuk menghasilkan soal penjumlahan dengan jumlah operand acak (2–5 angka) menggunakan perulangan for, sekaligus menampilkan soal dan meminta

jawaban pengguna. Jika jawaban benar, skor bertambah, jika salah, jumlah kesalahan meningkat dan jawaban yang benar ditampilkan.

Dari kode program di atas akan menghasilkan output:

```
2 + 4 + 9 + 1 + 10 = 26
9 + 1 + 6 = 16
7 + 6 + 1 + 4 + 3 = 15
Wrong! The answer was 21
10 + 7 + 7 + 3 + 8 = 35
8 + 10 + 9 + 3 = 25
Wrong! The answer was 30
8 + 9 = 17
5 + 8 + 9 + 9 = 36
Wrong! The answer was 31
You earned 4 total points.
```

Gambar 2.6 Output kode program “GamePenjumlahan”

2.5 Praktikum “SentinelLoop”

```
package pekan6_2511531007;

import java.util.Scanner;

public class SentinelLoop_2511531007 {

    public static void main(String[] args) {
        Scanner console = new Scanner (System.in);
        int sum = 0;
        int number = 12; //"dummy value", anything but 0

        while (number != 0) {
            System.out.print("Masukkan angka (0 untuk keluar): ");
            number = console.nextInt();
            sum = sum + number;
        }
        System.out.println("totalnya adalah " + sum);
    }
}
```

Gambar 2.7 kode program praktikum “SentinelLoop”

Program SentinelLoop menggunakan perulangan while dengan teknik sentinel loop untuk menjumlahkan deret angka yang dimasukkan pengguna, di mana proses berhenti ketika pengguna memasukkan nilai 0 sebagai penanda akhir. Variabel number diinisialisasi dengan nilai dummy (misalnya 12) agar kondisi loop pertama kali terpenuhi, lalu di dalam loop, program meminta input angka dan menambahkannya ke variabel sum. Perulangan akan terus berjalan selama number != 0, sehingga 0 berfungsi sebagai sentinel value, bukan bagian dari data yang

dijumlahkan, melainkan sinyal untuk menghentikan proses. Setelah loop selesai, program menampilkan total jumlah semua angka yang dimasukkan sebelumnya. Pendekatan ini sangat efisien untuk kasus di mana jumlah input tidak diketahui di awal dan bergantung pada interaksi pengguna.

Dari kode program di atas akan menghasilkan output:

```
Masukkan angka (0 untuk keluar): 30
Masukkan angka (0 untuk keluar): 50
Masukkan angka (0 untuk keluar): 100
Masukkan angka (0 untuk keluar): 0
totalnya adalah 180
```

Gambar 2.8 Output kode program “SentinelLoop”

2.6 Praktikum “doWhile”

```
package pekan6_2511531007;

import java.util.Scanner;

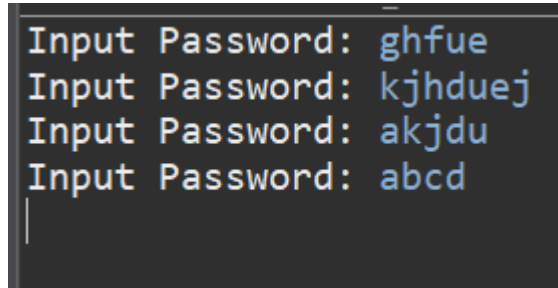
public class doWhile_2511531007 {

    public static void main(String[] args) {
        Scanner console = new Scanner (System.in);
        String phrase;
        do {
            System.out.print("Input Password: ");
            phrase = console.next();
        } while (!phrase.equals("abcd"));
    }
}
```

Gambar 2.8 kode program “doWhile”

Program ini menggunakan perulangan do-while untuk meminta pengguna memasukkan password hingga inputnya benar, dalam hal ini, password yang valid adalah "abcd". Struktur do-while dipilih karena memastikan blok kode dijalankan minimal satu kali, sehingga pengguna selalu diminta memasukkan password setidaknya sekali sebelum kondisi diperiksa. Setelah pengguna memasukkan input, program membandingkan nilai tersebut dengan password yang benar menggunakan metode `.equals()`, dan jika tidak cocok (`!phrase.equals("abcd")`), perulangan akan berlanjut. Proses ini terus berulang sampai password yang dimasukkan tepat, sehingga sangat cocok untuk validasi input yang membutuhkan minimal satu percobaan dan berhenti hanya saat syarat terpenuhi.

Dari kode program di atas akan menghasilkan output:



```
Input Password: ghfue
Input Password: kjhduej
Input Password: akjdu
Input Password: abcd
|
```

Gambar 2.9 Output kode program “doWhile”

BAB III

KESIMPULAN

3.1 Kesimpulan

Praktikum ini berhasil memperkuat pemahaman tentang berbagai bentuk perulangan dalam Java, khususnya while, do-while, dan integrasinya dengan logika kendali berbasis kondisi dinamis. Setiap program menunjukkan penerapan perulangan dalam konteks berbeda: GamePenjumlahan menggabungkan while dan for untuk mengelola alur permainan berbasis skor dan kesalahan; Lempardadu menggunakan while untuk simulasi acak yang berhenti saat kondisi spesifik (jumlah = 7) tercapai; SentinelLoop menerapkan pola sentinel value untuk menghentikan input berulang saat pengguna memasukkan nilai penanda (0); validasi password memanfaatkan do-while agar input selalu diminta minimal sekali sebelum diverifikasi; sedangkan PerulanganWhile1 menunjukkan penggunaan variabel flag untuk mengontrol kelanjutan program berdasarkan keputusan pengguna. Secara keseluruhan, praktikum ini menunjukkan bahwa perulangan bukan hanya alat untuk mengulang kode, tetapi juga fondasi penting dalam membangun program interaktif, responsif, dan adaptif terhadap input atau keadaan yang berubah-ubah, keterampilan esensial dalam pemrograman berbasis logika dan pengembangan perangkat lunak.

DAFTAR PUSTAKA

- [1] S. W. Tan, Pemrograman Berorientasi Objek dengan Java. Yogyakarta: Andi Publisher, 2021.
- [2] A. Kadir, Pengantar Java dan Pemrograman Berorientasi Objek (Edisi Revisi). Jakarta: PT Elex Media Komputindo, 2019.
- [3] S. Budhi dan A. Wibowo, Modul Praktikum Pemrograman Dasar. Yogyakarta: Laboratorium Ilmu Komputer, Universitas Gadjah Mada, 2023.
- [4] Oracle Corp., “Class Scanner,” Java Platform Standard Edition 21 API Specification, 2025. [Daring]. Tersedia: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Scanner.html>
- [5] Oracle Corp., “Class Random,” Java Platform Standard Edition 21 API Specification, 2025. [Daring]. Tersedia: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Random.html>